CoreThink for OpenClaw

A formal developer relations briefing for hybrid local-first reasoning systems in 2026

How to combine a reasoning-focused API with a self-hosted agent gateway, local memory, local tools, and deliberate model routing.

Bottom line. Treat CoreThink as a high-reasoning layer, not merely another chat endpoint. Treat OpenClaw as the local control plane: channels, memory, workspace, routing, isolation, and tool execution. The winning pattern is **reason remotely, act locally**. That design gives you stronger planning and tool selection without giving up a local-first system architecture.

Document type	DevRel briefing / implementation guide
Audience	Developers, infra-minded builders, local-first agent engineers, IDE / automation authors, and technical teams evaluating CoreThink inside an OpenClaw stack
Publication date	March 31, 2026
Positioning	Practical builder guide, not a whitepaper and not a vendor brochure

Validation note. Product-specific endpoint details, model names, and request parameters for CoreThink were reconciled against the supplied developer draft and public-facing CoreThink pages available on March 31, 2026. Public-facing materials did *not* surface a public token-pricing schedule for CoreThink during this review, so procurement and unit economics for CoreThink itself should be validated directly in the dashboard or with sales before external publication.

Prepared as a formal documentation briefing for immediate builder use.

Contents

1 Executive summary **3**

2 Who this briefing is for **3**
2.1 Best-fit developer profiles . . . 3
2.2 Who should *not* start here . . . 4

3 What CoreThink is, in practical engineering terms **4**
3.1 What the supplied product draft adds . . . 4

4 What OpenClaw contributes to the stack **4**
4.1 Local memory is explicit and inspectable . . . 5
4.2 Workspace and tools stay local . . . 5
4.3 Multi-agent routing is a first-class design feature . . . 5

5 The hybrid thesis: CoreThink as the reasoning brain, OpenClaw as the local control plane **5**
5.1 What should stay local . . . 5
5.2 What is worth sending to CoreThink . . . 5
5.3 Why this division works . . . 6

6 The five system patterns that matter most **6**
6.1 Pattern 1: Reason remotely, act locally . . . 6
6.2 Pattern 2: Local-first with remote escalation . . . 6
6.3 Pattern 3: Planner and executor split . . . 6
6.4 Pattern 4: Private context distillation . . . 6
6.5 Pattern 5: Reflective review loop . . . 6

7 What a developer should install around this stack: 30 open-source tools worth considering **6**

8 Quick comparison: CoreThink versus the common alternatives **9**
8.1 The practical difference to communicate in DevRel . . . 9

9 Cost model: local compute versus API credits **9**
9.1 What is knowable today . . . 9
9.2 What is *not* publicly clear . . . 10
9.3 Useful planning math . . . 10
9.4 The best economic design for most developers . . . 10

10 Security and operational constraints you should say out loud **11**
10.1 Prompt injection is still a system problem . . . 11
10.2 The workspace is not a hard sandbox by default . . . 11
10.3 The safe hybrid rule . . . 11
10.4 Recommended hardening checklist . . . 11

11 What you can build immediately **11**
11.1 1. Private repo architect . . . 11
11.2 2. Debugger with reflective review . . . 12

11.3 3. Release and incident copilot . . . 12
11.4 4. Research and synthesis assistant . . . 12
11.5 5. Smart contract or protocol analyst . . . 12
11.6 6. Multi-channel deep-work lane . . . 12

12 The tutorial: build a CoreThink-powered deep-work lane inside OpenClaw 12
12.1 Architecture target . . . 12
12.2 Step 1: smoke-test the CoreThink API directly . . . 12
12.3 Step 2: install OpenClaw and bring up onboarding . . . 13
12.4 Step 3: add CoreThink as a custom provider in onboarding . . . 13
12.5 Step 4: add a local model provider for cheap everyday turns . . . 14
12.6 Step 5: create two isolated agents . . . 14
12.7 Step 6: keep memory and repo state local . . . 15
12.8 Step 7: use the lane the right way . . . 15
12.9 Step 8: harden the system before you trust it . . . 16

13 What to publish externally versus what to keep as internal notes 16
13.1 Safe external messaging . . . 16
13.2 Internal-only items to validate before external release . . . 16

14 Final recommendations 16

A Sources and validation references 17

1 Executive summary

CoreThink is best understood as a reasoning-focused, OpenAI-compatible API surface layered on top of a model stack that is explicitly positioned for planning, code generation, and tool-calling. Public CoreThink materials frame the system around explainable multi-step workflows and a symbolic reasoning layer designed to improve tool-use generalization, while the supplied product draft positions the API as a drop-in replacement for OpenAI-style clients with support for tools, streaming, and multiple underlying models.[1, 2, 3, 4]

OpenClaw solves a different part of the stack. It is a self-hosted gateway that sits on your hardware, connects to the chat surfaces you already use, manages sessions and memory, routes work across multiple agents, and lets you run local tools and local models under your own operational rules. OpenClaw's own docs emphasize that it is agent-native, multi-channel, self-hosted, and built around explicit workspaces, local memory files, and isolated multi-agent routing.[5, 8, 10]

That makes the pairing compelling. You do not need CoreThink to replace your local system. You need CoreThink to make your local system better at the *hard parts*: planning, code reasoning, long-horizon problem decomposition, tool sequencing, and difficult synthesis. Meanwhile OpenClaw remains the place where your files, tools, channels, memory, and safety controls live.[3, 4, 11, 9]

Recommended architecture in one sentence. Keep memory, files, shells, channels, and safety policy local in OpenClaw; escalate only the difficult reasoning turns to CoreThink; then execute and persist results locally.

2 Who this briefing is for

This document is written for developers who want to feel at home quickly in a hybrid system rather than in a single-vendor cloud product.

2.1 Best-fit developer profiles

- **Local-first builder.** You want your files, notes, transcripts, prompts, and agent state to live on your hardware, but you still want access to stronger reasoning when a task becomes difficult.
- **Agent engineer.** You care about model routing, tool policies, long-running workflows, multi-agent design, evaluation loops, and failure modes.
- **Devtools and IDE integrator.** You want to wire advanced inference into editor extensions, coding copilots, repo assistants, or terminal workflows without rebuilding your whole stack.
- **Infra-minded developer.** You care about cost routing, provider abstraction, observability, security boundaries, and what happens when a hosted provider is slow or unavailable.
- **Security-conscious power user.** You want sandboxes, allowlists, local workspaces, explicit tool policy, and a controllable blast radius rather than an always-on cloud black box.
- **Web3 / technical automation builder.** You may care about chain data, smart contracts, or automation, but your real requirement is stronger multi-step reasoning and a practical deployment path.

2.2 Who should *not* start here

If your workload is mostly cheap, high-volume summarization or standard customer-support generation, a small hosted model or a simple local inference setup may be enough. This briefing is for builders who want more than chat: routing, memory, tools, execution, isolation, and difficult reasoning.

3 What CoreThink is, in practical engineering terms

Public-facing CoreThink pages present the product as a platform for explainable, multi-step AI workflows with API access, while the KiloCode integration guide shows that the product is exposed through an OpenAI-compatible base URL at `https://api.corethink.ai/v1/code`. That same integration guide instructs users to configure the provider as `OpenAI Compatible`, use the model name `corethink`, and authenticate with an API key created in the CoreThink dashboard.[1, 2]

The more technically revealing public material is the CoreThink research track. In the August 2025 public paper, CoreThink is described as a symbolic reasoning layer built to improve three concrete use cases: tool-calling, code generation, and planning. In the later MAVEN / agentic-reasoner write-up, CoreThink is described as a lightweight symbolic layer for structured decomposition, adaptive tool orchestration, and intermediate-result verification, with benchmark results reported as 5–30% over baselines at roughly one-tenth the computational cost in that evaluation setting.[3, 4]

That leads to a useful mental model:

> **A good builder's definition.** CoreThink is not just "another model." It is better framed as a reasoning-oriented inference layer that is especially useful when you already have an execution environment, tool graph, or agent runtime and want harder thinking without rewriting your client stack.

3.1 What the supplied product draft adds

The supplied draft extends the public picture with product-level API details: an OpenAI-style message format, tools / function calling, streaming, optional model selection, request controls such as `temperature` and `max_tokens`, and a set of named models including `gpt-oss-120b`, DeepSeek, Qwen, and Mistral variants. It also positions CoreThink as strong for planning-heavy agent workflows, code generation, and iterative development.

For DevRel purposes, that means you can document CoreThink at two levels simultaneously:

1. **Public positioning layer:** reasoning-first, explainable, tool-oriented, API-accessible.[1, 3]
2. **Builder compatibility layer:** OpenAI-style client surface, tool use, streaming, multi-model options, and easy insertion into existing developer tooling, as reflected in the supplied draft and the KiloCode integration page.[2]

4 What OpenClaw contributes to the stack

OpenClaw is a self-hosted gateway for AI agents that connects channels like WhatsApp, Telegram, Discord, iMessage, and others to a local agent runtime. It is designed around workspaces, sessions, multi-agent routing, and tool-enabled operation. Its docs explicitly position it as self-hosted, multi-channel, agent-native, and open source.[5]

Three parts of OpenClaw are especially important in a CoreThink-based architecture.

4.1 Local memory is explicit and inspectable

OpenClaw stores memory as plain Markdown files in the agent workspace. Its docs are unusually clear on this point: the system only remembers what is written to disk; there is no hidden state. Durable memory lives in `MEMORY.md`, while running context can be stored in date-scoped note files under `memory/YYYY-MM-DD.md`.[8]

4.2 Workspace and tools stay local

OpenClaw's workspace is the agent's local home directory for files and context. The docs also warn that the workspace is *not* a hard sandbox by default, which is critical when a remote model is allowed to invoke local tools. If you need isolation, you must enable sandboxing explicitly.[9]

4.3 Multi-agent routing is a first-class design feature

OpenClaw supports multiple isolated agents, each with its own workspace, auth, sessions, and routing rules. That makes it unusually well suited for hybrid designs in which one agent is optimized for everyday local work while another is reserved for "deep work" reasoning or higher-trust planning.[10]

5 The hybrid thesis: CoreThink as the reasoning brain, OpenClaw as the local control plane

The right way to explain this system to developers is not "cloud versus local." The right framing is **division of labor**. CoreThink should do the thinking that benefits from stronger inference. OpenClaw should do the work that benefits from local control.

Inputs and surfaces	Local control layer (OpenClaw)	Reasoning layer (CoreThink)
Telegram, Discord, WhatsApp, dashboard, local scripts, repo events	Workspace, sessions, memory files, bindings, tool policy, sandboxing, local runtimes, local files, model failover, channel routing	Planning, difficult code reasoning, long-horizon decomposition, tool sequencing, synthesis, review, critique

5.1 What should stay local

- Filesystem access, repo state, and shell execution.
- Persistent memory and daily notes.
- Channel identity, delivery, and routing logic.
- Sandboxing, allowlists, and tool-approval policy.
- Cheap repetitive tasks and fallback inference.

5.2 What is worth sending to CoreThink

- Hard planning and decomposition problems.
- Difficult refactors across multiple files.
- Ambiguous debugging and root-cause analysis.
- Review and critique passes on candidate solutions.

- Tool selection decisions when the path is not obvious.
- Any long-horizon problem where you want stronger chain planning but not full cloud ownership of the environment.

5.3 Why this division works

A pure cloud assistant is easy to start, but it owns too much of the system boundary. A pure local system is controllable, but difficult reasoning quality can become the limiting factor. The hybrid design gives you *local persistence and control* plus *selective frontier-grade reasoning.* OpenClaw's docs already endorse hybrid model configurations and fallbacks, while its local-model guidance explicitly recommends keeping hosted models configured even when you run local models, using `models.mode: "merge"`.[7]

6 The five system patterns that matter most

6.1 Pattern 1: Reason remotely, act locally

Use CoreThink to generate the plan, the constraints, the step order, or the review comments. Let OpenClaw execute the file operations, shell commands, and memory writes locally.

Best for: repo refactors, incident response, document-to-action workflows.

6.2 Pattern 2: Local-first with remote escalation

Default to a local model for everyday turns, but route known-hard tasks to a CoreThink-backed agent or fallback. This is the best default for cost-sensitive personal systems.

Best for: personal assistants, daily coding help, DM bots, notebook agents.

6.3 Pattern 3: Planner and executor split

Create one high-trust planner agent with limited execution capability and one executor agent with local tools and stricter sandboxing. OpenClaw's multi-agent isolation makes this natural.[10, 11]

Best for: security-sensitive automation and tool-enabled agents.

6.4 Pattern 4: Private context distillation

Keep raw private files local. Summarize only the relevant slices before sending a distilled packet to CoreThink. Store the result locally again.

Best for: sensitive repositories, contracts, internal docs, financial material.

6.5 Pattern 5: Reflective review loop

Generate locally or with a cheaper model; then send the output to CoreThink for critique, gap-finding, and plan repair.

Best for: code reviews, migration plans, architecture decisions, release notes, and risk checklists.

7 What a developer should install around this stack: 30 open-source tools worth considering

The list below is intentionally opinionated. It emphasizes open-source tools that either improve local inference, routing, agent construction, memory, observability, or security for a CoreThink + OpenClaw workflow.

Why this list excludes LM Studio. OpenClaw's local-model docs currently recommend LM Studio for a high-end local stack, but LM Studio is not open source, so it is excluded from the curated OSS list below even though it remains a pragmatic option for some developers.[7]

Tool	Category	Why it matters in a CoreThink + OpenClaw system
OpenClaw	Gateway / runtime	Your self-hosted control plane for channels, sessions, workspaces, memory, and routing. It is the backbone of the hybrid system.
Ollama	Local runtime	Easiest low-friction local model runtime; good for experimentation, fallbacks, and small always-on assistants.
llama.cpp	Local runtime	Excellent when you want maximal portability, quantized open models, and fine control over consumer hardware deployment.
vLLM	Serving	Strong choice for serious local or private-cluster serving behind an OpenAI-compatible endpoint.
SGLang	Serving	Useful for higher-performance serving and specialized inference pipelines when you are tuning throughput.
LocalAI	Serving	Lets you expose a local OpenAI-style endpoint over multiple model backends with minimal glue.
Text Generation Inference (TGI)	Serving	Hugging Face's serving stack for larger open models and team-grade deployments.
TensorRT-LLM	Serving	Worth considering for NVIDIA-heavy teams optimizing latency and throughput on dedicated GPU infrastructure.
LiteLLM	Gateway / routing	Unified proxy for model routing, fallback, provider abstraction, and usage instrumentation across many APIs.
FastAPI	App layer	Excellent for small local bridge services, evaluation harnesses, and custom tool endpoints around OpenClaw.
FastMCP	Protocol / tools	Simple way to expose custom capabilities as MCP services for agents and tool-aware applications.
OpenHands	Agent framework	Full-stack autonomous software engineering environment for heavier automation and task execution.
Aider	Coding agent	Terminal-native pair programming that works well as a local execution surface or review assistant.
Continue	IDE integration	Open-source editor assistant with strong customization for local and remote model combinations.
Cline	IDE integration	Popular agentic coding workflow in VS Code, useful when you want tool use and iterative execution loops.
Roo Code	IDE integration	Another strong open-source coding assistant option for builders who want editable prompts and provider control.
LangGraph	Agent orchestration	Good when your workflow needs explicit state machines, branching, retries, or durable agent graphs.
AutoGen	Multi-agent framework	Helpful for experiments in planner / critic / executor agent splits.
CrewAI	Multi-agent framework	Useful if you want role-based collaboration patterns without building orchestration from scratch.
Qdrant	Vector store	Strong default for semantic search and memory retrieval in local or server deployments.
Chroma	Vector store	Lightweight option for small local projects and prototypes.
Weaviate	Vector database	Good when your retrieval layer needs scale, filters, and a more feature-rich database.

Tool	Category	Why it matters in a CoreThink + OpenClaw system
LanceDB	Retrieval	Fast local or embedded vector storage with a developer-friendly feel.
Milvus	Retrieval	Better suited to larger-scale retrieval infrastructure or heavier internal search workloads.
RAGFlow	Retrieval pipeline	Helpful when you want a more end-to-end document ingestion and RAG workflow.
Mem0	Memory layer	Useful when you want structured memory management on top of agent interactions.
Semgrep	Security / code analysis	Fast static analysis for local repos, especially useful before or after tool-enabled code modifications.
Tree-sitter	Code structure	Gives your local system better syntax awareness for chunking, diffing, and repo navigation.
ripgrep	Local search	Still one of the fastest and most practical ways to make a local code agent more effective.
OpenTelemetry	Observability	Essential once your hybrid system has multiple hops and you need traces, spans, or runtime diagnostics.
Podman	Isolation	Strong containerization option for sandboxing and running tools with less host exposure.
uv	Python environment management	Fast way to bootstrap reproducible Python-based tools, bridges, and evaluation scripts.

8 Quick comparison: CoreThink versus the common alternatives

Option	Best at	What you manage	Strongest advantage	Main tradeoff
CoreThink + OpenClaw hybrid	Hard reasoning with local control	Local gateway, memory, routing, tool policy, plus one remote reasoning provider	Better system design for builders who want privacy, channels, local state, and stronger reasoning only when it matters	More moving parts than a single hosted API
OpenAI API only	Broadest general platform and tool surface	Mostly prompts, app code, and billing	Mature SDKs, public pricing, broad tool catalog, and strong frontier models for agents and coding	Less local ownership; your app has to recreate memory, routing, and channel logic itself
Anthropic API only	Long-running coding and agent workflows	Mostly prompts, app code, and billing	Strong agentic coding profile with clear public pricing for Sonnet and Opus	Still a hosted-first model; local system control remains your job
Local open models only	Maximum control and no per-token billing	Hardware, serving, quantization, security posture, routing, and reliability	Best privacy and no external token meter	Quality, latency, hardware burden, and prompt-injection robustness can be limiting
Managed gateway + local tools	Fast multi-provider experimentation	Gateway config and provider selection	Easy provider swapping and unified billing / routing patterns	Still does not automatically give you local memory architecture or tool-safe execution patterns

8.1 The practical difference to communicate in DevRel

OpenAI is the most complete public developer platform in this comparison: public pricing, Responses API, agent tools, and a broad model catalog. Anthropic is especially strong for long-running coding and agent workflows, with clear Sonnet and Opus positioning. The reason to tell a different story for CoreThink is not "because it is another model vendor." The story is that CoreThink's public research and product framing are unusually aligned around *reasoning structure, tool orchestration, and planning.* That makes it especially relevant when dropped into an existing local agent environment rather than used as a standalone chatbot.[3, 4, 13, 14, 15]

9 Cost model: local compute versus API credits

9.1 What is knowable today

OpenClaw's local-model guidance is blunt: for strong local tool-enabled agents with large context and reasonable prompt-injection resistance, you should aim high. The docs describe a serious local setup as at least two maxed-out Mac Studios or an equivalent GPU rig, roughly $30k+, and warn that a single 24 GB GPU is only viable for lighter prompts with higher latency.[7]

On the hosted side, OpenAI publishes current model pricing directly. As of March 31, 2026, GPT-5.4 is listed at $2.50 per million input tokens and $15 per million output tokens, while GPT-5.4 mini is listed at $0.75 per million input and $4.50 per million output. Anthropic lists Claude Sonnet 4.6 at $3 / $15 per million input/output tokens and Claude Opus 4.6 at $5 / $25.[13, 14, 15]

9.2 What is *not* publicly clear

We did not locate a public CoreThink token-pricing page during this review. That means you should **not** publish token-level cost claims for CoreThink until you confirm them from the live dashboard, a sales contact, or a formal pricing page.

9.3 Useful planning math

The table below uses OpenAI's public pricing only as a reference benchmark for hosted inference economics. It is *not* a proxy quote for CoreThink.

Scenario	Assumption	Approx. cost	Interpretation
High-end local stack	$30,000 amortized over 36 months	$833 / month before power, cooling, and ops	A strong local stack can be economical only if you keep it busy enough to justify the capital and operational burden
Light routing call on GPT-5.4 mini	2k input / 400 output	$0.0033	Cheap enough that remote reasoning for selective triage or routing can be very affordable
Code review call on GPT-5.4 mini	20k input / 3k output	$0.0285	A meaningful but still modest cost per heavier review or critique pass
Heavy planning call on GPT-5.4	8k input / 2k output	$0.05	Hosted deep-thinking calls remain cheap if volume is low and the work is high value
Heavy coding call on GPT-5.4	20k input / 3k output	$0.095	Expensive enough to matter at scale, but still often cheaper than underutilized local hardware

At $833 per month of amortized local capital, you could instead buy approximately 16,666 GPT-5.4 "heavy planning" calls per month, or roughly 252,525 GPT-5.4 mini "light routing" calls per month, under the illustrative assumptions above.

Decision rule. If your workload is bursty, mainly interactive, or only occasionally hard, remote reasoning is usually cheaper and simpler. If your workload is steady, high-volume, sensitive, and fully local by requirement, local inference becomes more attractive — but you still may want to reserve CoreThink for the hardest cases.

9.4 The best economic design for most developers

For most individual developers and small teams, the sweet spot is:

1. **Local default** for chatty, repetitive, low-risk, or cheap work.

2. **CoreThink escalation** for the turns where better reasoning changes the outcome.
3. **Local execution and memory** for all durable artifacts and operational control.

That architecture avoids the worst of both extremes: oversized local capex on every turn, or expensive hosted reasoning on work a local model could handle.

10 Security and operational constraints you should say out loud

This document should be candid with developers. Hybrid systems are powerful, but they expand the attack surface unless you design boundaries explicitly.

10.1 Prompt injection is still a system problem

OpenClaw's security docs say this plainly: prompt injection is not solved by system prompts, and hard enforcement comes from tool policy, execution approvals, sandboxing, and channel allowlists. The same docs also recommend strong latest-generation models for tool-enabled agents and warn that smaller or older models are more susceptible to instruction hijacking and tool misuse.[11]

10.2 The workspace is not a hard sandbox by default

If you let a hosted reasoning provider drive local tools, you must enable sandboxing or otherwise constrain execution. OpenClaw explicitly states that the workspace is the default current working directory, not a hard sandbox.[9]

10.3 The safe hybrid rule

Safe default. Do not give the CoreThink-facing agent unrestricted host execution. Give it narrow tools, distilled context, and a sandboxed or separate executor path. Keep secrets out of the agent-visible filesystem whenever possible.[11, 9]

10.4 Recommended hardening checklist

- Keep inbound channels on pairing / allowlists rather than open DM policies.[11]
- Prefer mention gating in groups.[11]
- Use a planner / executor split for untrusted content or high-risk tools.[10, 11]
- Keep memory, credentials, and sessions local and private.[8, 9]
- Use a strong model tier for any agent that reads untrusted content or touches files / networks.[11]
- Treat web pages, email, pasted logs, and documents as hostile inputs by default.[11]

11 What you can build immediately

11.1 1. Private repo architect

A local coding assistant that indexes your codebase locally, stores preferences in OpenClaw memory, and escalates architecture planning or large refactor design to CoreThink.

11.2 2. Debugger with reflective review

Your local agent reproduces failures and gathers logs; CoreThink reviews the evidence, suggests fault trees, and critiques candidate fixes.

11.3 3. Release and incident copilot

OpenClaw watches local repo state, deployment notes, or CI events; CoreThink turns raw signals into prioritized next actions, rollback advice, and incident summaries.

11.4 4. Research and synthesis assistant

The local system collects and normalizes documents; CoreThink performs deeper synthesis, gap analysis, or comparison work; findings are saved back to local memory and docs.

11.5 5. Smart contract or protocol analyst

Local tooling pulls code, tests, traces, or chain data; CoreThink handles higher-level reasoning, planning, and structured review.

11.6 6. Multi-channel deep-work lane

Keep a cheap daily assistant on one channel and route a dedicated "deep work" channel or peer to a CoreThink-backed agent for only the difficult tasks. OpenClaw's routing model makes this clean.[10]

12 The tutorial: build a CoreThink-powered deep-work lane inside OpenClaw

This tutorial gives you a complete first build with a useful outcome:

- OpenClaw runs locally as your control plane.
- A default agent handles everyday work.
- A second agent is reserved for higher-value reasoning and uses CoreThink.
- Memory, routing, and durable artifacts remain local.
- You choose when to escalate into the "deep work" lane.

12.1 Architecture target

Desired end state. Everyday tasks stay on a low-cost local or fallback model. Hard planning, design, and code review are routed to a dedicated CoreThink agent. Both agents live behind one OpenClaw Gateway, but keep separate workspaces and sessions.

12.2 Step 1: smoke-test the CoreThink API directly

Before wiring anything into OpenClaw, confirm the API works from a normal client. CoreThink's public KiloCode guide uses the base URL `https://api.corethink.ai/v1/code` and the model name `corethink`.[2]

Listing 1: Direct Python smoke test using the OpenAI SDK interface

```
from openai import OpenAI
import os
```

```
client = OpenAI(
    base_url="https://api.corethink.ai/v1/code",
    api_key=os.environ["CORETHINK_API_KEY"],
)

resp = client.chat.completions.create(
    model="corethink",  # If your org exposes explicit model IDs, swap them in here.
    messages=[
        {"role": "system", "content": "You are a planning assistant for local agent
    workflows."},
        {"role": "user", "content": "Design a safe planner/executor split for a local
    coding assistant."},
    ],
    temperature=0.2,
)

print(resp.choices[0].message.content)
```

If this works, you have already validated the hardest external dependency.

12.3 Step 2: install OpenClaw and bring up onboarding

OpenClaw's quick start is straightforward: install the package, run onboarding, and optionally install the daemon.[5, **?**, 6]

Listing 2: Install and onboard OpenClaw

```
npm install -g openclaw@latest
openclaw onboard --install-daemon
openclaw dashboard
```

You will need Node 24 recommended, or Node 22.14+ for compatibility.[5, **?**]

12.4 Step 3: add CoreThink as a custom provider in onboarding

OpenClaw's onboarding overview explicitly supports custom or unlisted providers. Choose **Custom Provider** and enter:[6]

- **Compatibility mode:** OpenAI-compatible
- **Base URL:** `https://api.corethink.ai/v1/code`
- **API key:** your `CORETHINK_API_KEY`
- **Model ID:** start with `corethink`
- **Alias:** optional, for example `Reasoner`

After onboarding completes, run:

```
openclaw models list
```

Copy the exact model reference that OpenClaw created for the custom provider. Use *that* model reference in later config rather than guessing the provider prefix.

12.5 Step 4: add a local model provider for cheap everyday turns

OpenClaw's local-model docs recommend keeping hosted models available even when running local. The docs also provide an exact LM Studio provider block using `models.mode: "merge"`.[7]

Listing 3: Representative local provider block from the OpenClaw local-model pattern

```
{
  models: {
    mode: "merge",
    providers: {
      lmstudio: {
        baseUrl: "http://127.0.0.1:1234/v1",
        apiKey: "lmstudio",
        api: "openai-responses",
        models: [
          {
            id: "my-local-model",
            name: "Local Model",
            reasoning: false,
            input: ["text"],
            cost: { input: 0, output: 0, cacheRead: 0, cacheWrite: 0 },
            contextWindow: 196608,
            maxTokens: 8192,
          },
        ],
      },
    },
  },
}
```

If you prefer another local server, OpenClaw also documents vLLM and other OpenAI-compatible local proxies.[12, 7]

12.6 Step 5: create two isolated agents

OpenClaw's multi-agent design treats each agent as a separate brain with its own workspace, sessions, and auth profiles. Use that. Do not try to cram local chat and deep reasoning into one monolithic agent.[10]

```
openclaw agents add chat
openclaw agents add deepwork
openclaw agents list --bindings
```

Then edit `/.openclaw/openclaw.json` so the everyday agent uses the local model and the deep-work agent uses the CoreThink model reference discovered earlier.

Listing 4: Practical two-agent routing pattern

```
{
  agents: {
    list: [
      {
        id: "chat",
        name: "Everyday",
```

```
        workspace: "~/.openclaw/workspace-chat",
        model: "lmstudio/my-local-model",
      },
      {
        id: "deepwork",
        name: "Deep Work",
        workspace: "~/.openclaw/workspace-deepwork",
        model: "<replace-with-your-corethink-model-ref>",
      },
    ],
  },
  bindings: [
    { agentId: "chat", match: { channel: "telegram", accountId: "default" } },
    { agentId: "deepwork", match: { channel: "telegram", accountId: "deepwork" } },
  ],
}
```

This pattern mirrors OpenClaw's documented multi-agent routing examples: different agents, separate workspaces, distinct bindings, and deterministic routing.[10]

12.7 Step 6: keep memory and repo state local

Store durable preferences and project facts in `MEMORY.md`; let daily task context accumulate in dated note files. OpenClaw already loads those locally and makes them searchable.[8]

For example, put items like these in the deep-work agent's workspace:

```
MEMORY.md
AGENTS.md
TOOLS.md
memory/2026-03-31.md
repo-notes/
architectures/
```

12.8 Step 7: use the lane the right way

The best operating pattern is:

1. Ask the `chat` agent to gather, read, search, or summarize locally.
2. Forward the distilled problem statement to `deepwork` when you need planning, critique, or hard reasoning.
3. Save the answer back into local files or memory.
4. Let local tools or the local agent perform the actual file and shell work.

Example prompts:

- `chat`: "Summarize the repo layout and open bug reports in 12 bullets."
- `deepwork`: "Given this repo summary, design a safe migration plan from REST to async workers. Identify risk, test order, and rollback path."

- `chat`: "Turn that plan into concrete git tasks and stage the first file changes in a sandbox."

12.9 Step 8: harden the system before you trust it

Use OpenClaw sandboxing, allowlists, and limited tool policies for any agent that touches untrusted content or local execution. If the deep-work agent is remote-backed, keep its tool surface narrower than the local executor's. OpenClaw's security docs are clear that prompt injection is not solved by prompt text alone.[11, 9]

> **Practical rule.** Planner agents should be smart and narrow. Executor agents should be local and constrained. Do not combine "maximum intelligence" and "maximum host access" in one place unless you truly trust the full content path.

13 What to publish externally versus what to keep as internal notes

13.1 Safe external messaging

For DevRel publication, lead with:

- CoreThink as a reasoning-first API that fits existing OpenAI-style client habits.
- OpenClaw as the self-hosted local control plane.
- Hybrid architecture as the core benefit.
- Cost routing and security boundaries as deliberate design advantages.

13.2 Internal-only items to validate before external release

- Publicly quotable CoreThink pricing.
- Final public model catalog and naming conventions.
- Any claim about privacy, retention, or training use specific to the API and enterprise terms.
- Any claim about observable reasoning traces that is stronger than what the public research and docs support.

14 Final recommendations

1. Do not document CoreThink as just another endpoint. Document it as a **reasoning tier**.[3, 4]
2. Do not document OpenClaw as merely a chat bridge. Document it as a **local agent operating environment**.[5, 8, 10]
3. Recommend **local-first with remote escalation** as the default deployment pattern for individual developers and small teams.
4. Be explicit about the trust boundary: local memory and execution should remain local; remote reasoning should be selective.
5. Publish one simple tutorial — like the deep-work lane above — instead of ten shallow snippets.

The one-sentence takeaway for developers. If you already like building locally, CoreThink does not replace that instinct; it upgrades it. OpenClaw gives you the hands, memory, and environment. CoreThink gives you a stronger brain for the moments that matter most.

A Sources and validation references

References

[1] CoreThink AI, "CoreThink.ai," official homepage, crawled March 2026. https://corethink.ai/

[2] CoreThink AI, "KiloCode Integration," official docs / search result snippet showing OpenAI-compatible configuration, base URL, API key flow, and model name, crawled March 2026. https://corethink.ai/docs/kilocode

[3] Jay Vaghasiya, Omkar Ghugarkar, Vishvesh Bhat, Vipul Dholaria, Julian McAuley, "CoreThink: A Symbolic Reasoning Layer to reason over Long Horizon Tasks with LLMs," arXiv:2509.00971, 2025. https://arxiv.org/abs/2509.00971

[4] CoreThink AI, "On Generalization in Agentic Tool Calling / MAVEN dataset write-up," official blog page and search excerpt describing the agentic reasoner, 5–30% gains over baselines, and roughly one-tenth compute cost in that evaluation setting, 2025. https://corethink.ai/blog/corethink-maven-dataset

[5] OpenClaw Documentation, "OpenClaw" and "Getting Started," official docs, crawled March 2026. https://docs.openclaw.ai/ and https://docs.openclaw.ai/start/getting-started

[6] OpenClaw Documentation, "Onboarding Overview," official docs, crawled March 2026. https://docs.openclaw.ai/start/onboarding-overview

[7] OpenClaw Documentation, "Local Models," official docs, crawled March 2026. https://docs.openclaw.ai/gateway/local-models

[8] OpenClaw Documentation, "Memory Overview," official docs, crawled March 2026. https://docs.openclaw.ai/concepts/memory

[9] OpenClaw Documentation, "Agent Workspace," official docs, crawled March 2026. https://docs.openclaw.ai/concepts/agent-workspace

[10] OpenClaw Documentation, "Multi-Agent Routing," official docs, crawled March 2026. https://docs.openclaw.ai/concepts/multi-agent

[11] OpenClaw Documentation, "Security," official docs, crawled March 2026. https://docs.openclaw.ai/gateway/security

[12] OpenClaw Documentation, "vLLM," official docs, crawled March 2026. https://docs.openclaw.ai/providers/vllm

[13] OpenAI, "Models" and "Compare models," official API documentation, crawled March 2026. https://developers.openai.com/api/docs/models and https://developers.openai.com/api/docs/models/compare

[14] Anthropic, "Claude Sonnet 4.6," official model page, crawled March 2026. https://www.anthropic.com/claude/sonnet

[15] Anthropic, "Claude Opus 4.6," official model page, crawled March 2026. https://www.anthropic.com/claude/opus